

SmartLogix

Análisis de Patrones de Diseño y Arquetipos

Evaluación Parcial N°2 — Experiencia de Aprendizaje

Patrones de Diseño

Arquetipos Maven

Arquitectura Microservicios

BFF

Institución: Duoc UC

Fecha: Abril 2026

Versión: 1.0.0

1. Introducción

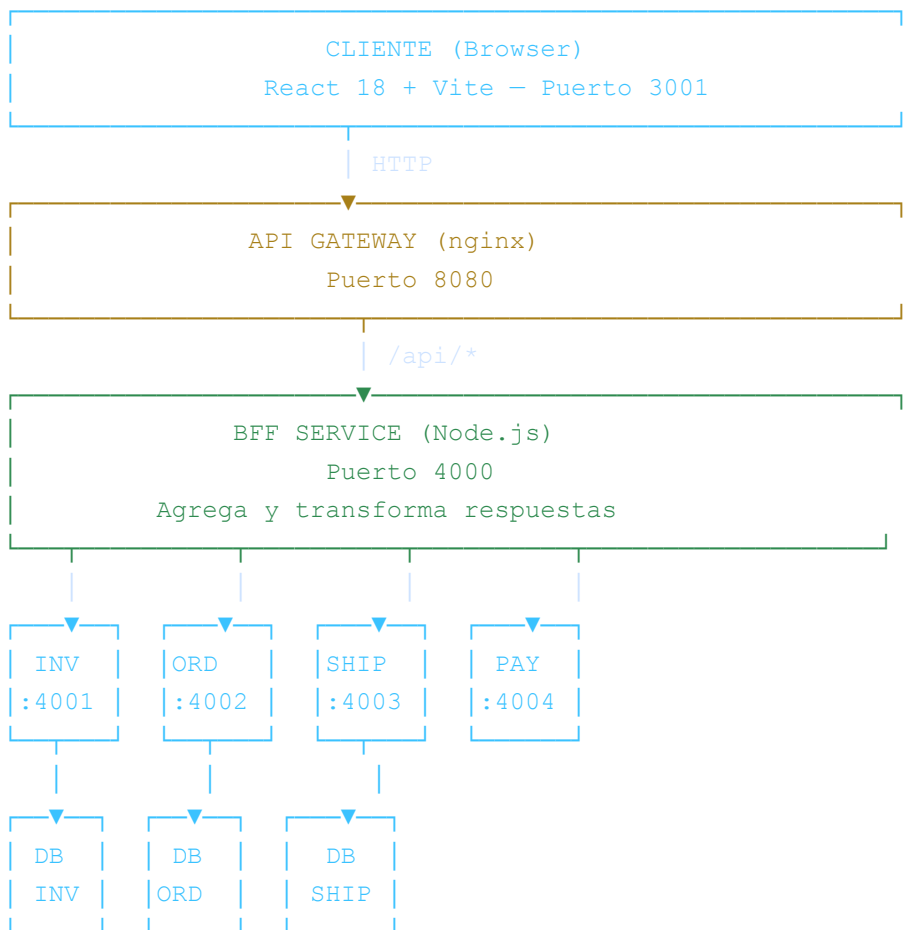
SmartLogix es un sistema de gestión logística desarrollado con una arquitectura de microservicios. El sistema está compuesto por un frontend React, un Backend For Frontend (BFF) y cuatro microservicios independientes: inventario, pedidos, envíos y pagos.

Este documento describe los patrones de diseño y arquetipos seleccionados para su implementación, justificando cada elección en función de los requerimientos funcionales y no funcionales del sistema.

Objetivo: Demostrar cómo los patrones de diseño y los arquetipos Maven contribuyen a una arquitectura modular, mantenible y escalable para SmartLogix.

2. Arquitectura General del Sistema

SmartLogix implementa el patrón arquitectónico de **Microservicios** con un **API Gateway** centralizado y un **Backend For Frontend** que agrega y transforma las respuestas para el cliente web.



Componente	Tecnología	Puerto	Responsabilidad
Frontend	React 18 + Vite	3001	Interfaz de usuario
API Gateway	nginx	8080	Enrutamiento y rate limiting
BFF Service	Node.js + Express	4000	Agregación para el frontend
Inventory Service	Node.js + PostgreSQL	4001	Productos y stock
Orders Service	Node.js + PostgreSQL	4002	Pedidos y estados
Shipping Service	Node.js + PostgreSQL	4003	Envíos y tracking
Payment Service	Node.js + Stripe/MP	4004	Procesamiento de pagos

3. Patrones Arquitectónicos

Microservicios

PATRÓN ARQUITECTÓNICO

Descripción:	El sistema se divide en servicios independientes, cada uno con su propia base de datos y ciclo de despliegue.
Dónde:	inventory-service, orders-service, shipping-service, payment-service.
Justificación:	Permite escalar cada servicio de forma independiente. El servicio de inventario puede tener más réplicas que el de pagos sin afectar al resto.
Beneficio:	Desacoplamiento, tolerancia a fallos parciales, despliegues independientes.

Backend For Frontend (BFF)

PATRÓN ARQUITECTÓNICO

Descripción:	Capa intermedia entre el cliente web y los microservicios, que adapta y agrega datos específicamente para la interfaz.
Dónde:	bff-service (Puerto 4000).
Justificación:	El frontend necesita datos de múltiples servicios en una sola pantalla (ej. dashboard combina KPIs de inventario, pedidos y envíos). Sin BFF, el cliente haría N llamadas paralelas aumentando la complejidad.
Beneficio:	Reduce el acoplamiento del frontend con los microservicios, permite evolucionar la API sin romper el cliente.

API Gateway

PATRÓN ARQUITECTÓNICO

Descripción:	Punto único de entrada al sistema que maneja enrutamiento, CORS, rate limiting y TLS.
Dónde:	nginx (Puerto 8080).
Justificación:	Evita exponer directamente los puertos internos de cada microservicio. Centraliza políticas de seguridad y logging.
Beneficio:	Seguridad centralizada, rate limiting de 30 req/s, registro de todas las solicitudes.

4. Patrones de Diseño

4.1 Patrones Creacionales

Factory Method

PATRÓN CREACIONAL (GOF)

- Descripción:** Define una interfaz para crear objetos, pero delega en subclases la decisión de qué clase instanciar.
- Dónde:** `inventory-service/src/factories/productFactory.js`
- Aplicación:** La fábrica genera productos según su tipo (físico, digital, perecedero), asignando automáticamente el SKU con prefijo correspondiente (PHY-, DIG-, PER-) y validando las reglas de negocio de cada tipo.
- Justificación:** Sin este patrón, la lógica de creación estaría dispersa en el controlador, dificultando agregar nuevos tipos de producto en el futuro.

4.2 Patrones Estructurales

Repository Pattern

PATRÓN ESTRUCTURAL / ACCESO A DATOS

- Descripción:** Abstrae el acceso a la capa de persistencia, separando la lógica de negocio de las consultas a la base de datos.
- Dónde:** `inventory-service/src/repositories/inventoryRepository.js`
`orders-service/src/repositories/`
`shipping-service/src/repositories/`
- Aplicación:** Los controladores no acceden directamente a los modelos Sequelize. Toda interacción con la base de datos pasa por el repositorio correspondiente.
- Justificación:** Facilita el testing (se puede reemplazar el repositorio por un mock), y permite cambiar el ORM sin tocar los controladores.

4.3 Patrones de Comportamiento

Circuit Breaker

PATRÓN DE COMPORTAMIENTO / RESILIENCIA

Descripción:	Detecta fallos repetidos en llamadas a servicios externos y "abre el circuito" para evitar cascada de errores.
Dónde:	<code>inventory-service/src/middleware/circuitBreaker.js</code>
Aplicación:	Si un microservicio dependiente no responde en el tiempo configurado, el Circuit Breaker retorna un error controlado inmediatamente en lugar de esperar un timeout largo.
Justificación:	En una arquitectura de microservicios, un servicio caído no debe bloquear todo el sistema. Este patrón garantiza resiliencia y tiempo de respuesta predecible.

Interceptor (Axios)

PATRÓN DE COMPORTAMIENTO

Descripción:	Intercepta solicitudes y respuestas HTTP para aplicar transformaciones transversales.
Dónde:	<code>frontend/src/services/api.js</code>
Aplicación:	El interceptor de respuesta desenvuelve automáticamente el objeto <code>res.data</code> , extrae mensajes de error de validación y rechaza la promesa con un mensaje legible.
Justificación:	Evita duplicar el manejo de errores en cada llamada API del frontend. Un solo lugar centraliza la transformación de respuestas.

5. Arquetipos Maven

Los arquetipos Maven son plantillas de proyecto que permiten generar estructuras base estandarizadas con un único comando. Para SmartLogix se diseñaron tres arquetipos que reflejan los distintos tipos de componentes backend del sistema.

5.1 archetype-bff

Arquetipo: Backend For Frontend

SPRING BOOT 3 + WEBFLUX + ACTUATOR

Genera:	Aplicación Spring Boot con BffController y cliente WebFlux para llamar microservicios.
Parámetros:	serviceName, servicePort
Uso típico:	Crear un BFF Java cuando el equipo migra de Node.js a Spring Boot.
Comando:	<code>mvn archetype:generate -DarchetypeArtifactId=archetype-bff ...</code>

5.2 archetype-microservice-jpa

Arquetipo: Microservicio con Persistencia JPA

SPRING BOOT 3 + SPRING DATA JPA + POSTGRESQL + LOMBOK

Genera:	Microservicio completo con Entity, Repository (Spring Data), Controller con CRUD completo y health check.
Parámetros:	serviceName, servicePort, entityName
Uso típico:	Servicios que gestionan datos propios: inventario, pedidos, envíos.
Patrón interno:	Repository Pattern implementado con Spring Data JPA ({Entity}Repository extends JpaRepository).

5.3 archetype-microservice-simple

Arquetipo: Microservicio REST Ligero

SPRING BOOT 3 + SPRING WEB + WEBFLUX (CLIENTE)

Genera:	Microservicio sin base de datos propia, con un controlador REST base y cliente WebFlux para integrar APIs externas.
Parámetros:	serviceName, servicePort
Uso típico:	Servicios de integración: pagos (Stripe/MercadoPago), notificaciones, autenticación externa.

5.4 Relación entre Arquetipos y Microservicios SmartLogix

Microservicio	Arquetipo base	Justificación
bff-service	archetype-bff	Agrega respuestas de múltiples servicios para el frontend
inventory-service	archetype-microservice-jpa	Gestiona productos, stock y bodegas en PostgreSQL
orders-service	archetype-microservice-jpa	Gestiona pedidos con estado persistente en PostgreSQL
shipping-service	archetype-microservice-jpa	Gestiona envíos con tracking en PostgreSQL
payment-service	archetype-microservice-simple	Integra APIs externas (Stripe, MercadoPago), sin BD propia

6. Resumen de Patrones Aplicados

Patrón	Categoría	Componente
Microservicios	Arquitectónico	Sistema completo
Backend For Frontend (BFF)	Arquitectónico	bff-service
API Gateway	Arquitectónico	nginx
Factory Method	Creacional (GoF)	inventory-service
Repository	Estructural / Datos	inventory, orders, shipping
Circuit Breaker	Comportamiento / Resiliencia	inventory-service
Interceptor	Comportamiento	frontend (Axios)
Rate Limiting	Estabilidad	nginx + bff-service

Conclusión: La combinación de patrones arquitectónicos (Microservicios + BFF + API Gateway) con patrones de diseño (Factory, Repository, Circuit Breaker) permite a SmartLogix ser un sistema modular, resiliente y preparado para escalar. Los arquetipos Maven estandarizan la creación de nuevos componentes, reduciendo el tiempo de configuración inicial y asegurando consistencia entre servicios.